

assigned to the `purse` and `coinJar` variables. These methods calculate values based on the object's `galleons`, `sickles`, and `knuts` attributes.

Classes and OOP can lead to more *maintainable* code—that is, code that is easier to read, modify, and extend in the future. Let's explore this class's methods and attributes in more detail.

Methods, `__init__()`, and `self`

Methods are functions associated with objects of a particular class. Recall that `lower()` is a string method, meaning that it's called on string objects. You can call `lower()` on a string, as in `'Hello'.lower()`, but you can't call it on a list, such as `['dog', 'cat'].lower()`. Also, notice that methods come after the object: the correct code is `'Hello'.lower()`, not `lower('Hello')`. Unlike a method like `lower()`, a function like `len()` is not associated with a single data type; you can pass strings, lists, dictionaries, and many other types of objects to `len()`.

As you saw in the previous section, we create objects by calling the class name as a function. This function is referred to as a *constructor function* (or *constructor*, or abbreviated as *ctor*, pronounced “see-tore”) because it constructs a new object. We also say the constructor *instantiates* a new instance of the class.

Calling the constructor causes Python to create the new object and then run the `__init__()` method. Classes aren't required to have an `__init__()` method, but they almost always do. The `__init__()` method is where you commonly set the initial values of attributes. For example, recall that the `__init__()` method of `WizCoin` looks like the following:

```
def __init__(self, galleons, sickles, knuts):
    """Create a new WizCoin object with galleons, sickles, and knuts."""
    self.galleons = galleons
    self.sickles  = sickles
    self.knuts   = knuts
    # NOTE: __init__() methods NEVER have a return statement.
```

When the `wcexample1.py` program calls `WizCoin(2, 5, 99)`, Python creates a new `WizCoin` object and then passes three arguments (2, 5, and 99) to an `__init__()` call. But the `__init__()` method has four parameters: `self`, `galleons`,